PYTHON FOR VIDEO GAMES WITH PYGAME · LESSON 5

Collisions, score and lives

Goal: By the end you can detect collisions between Rects, add points, show text on screen and take away lives.
Before you start:

- Lesson 4 - the falling enemies.

To copy the code: when you copy from a PDF, many viewers drop the spaces at the start of each line, and in Python those spaces matter. Every step is also a `.py` file in the same folder as this guide: open it and copy from there.

Step by step

Collisions, score and lives

So far the bullets go through the enemies; today that ends. You will detect when a bullet touches an enemy. You will add points, and show them on screen. And you will lose lives when an enemy gets past you or crashes into you.

Step 2

A collision is just this: two rectangles touching. First, below the shot sound, load hit_sound, the sound that will play when a bullet hits. After creating the enemy group, add the variable score, set to zero. That is where the points for the whole game will add up. And after moving the enemies, go over the group with a for loop, and ask each one whether its Rect touches any bullet. collidelist answers with that bullet's position in the list, or minus one if it touches none; that is why the if compares with minus one. If it hits, bullets.pop(hit) removes that bullet, so it does not count twice. Reset sends the enemy back up, you add ten points, and the hit sound plays.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-1.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
```

```
class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
```

```
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 3

Run the game. Now each bullet that hits makes the enemy disappear, and you hear the hit. The points are already adding up, in the score variable, but you cannot see them yet.

Try it:

```
python game.py
```

Step 4

To write on the screen, you first need a font. pygame.font.Font(None, 36) creates it once, at the start: None means the font that comes with pygame, and thirty-six is the size. At the end of the drawing, before display.flip, font.render turns the text into an image, because pygame cannot draw text directly. The f before the quotes and the braces around score put its value inside the text, True smooths the edges, and the last value is the color white. With blit you paste that image in the top-left corner, every frame.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-2.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)
```

```
    def update(self):
        self.rect.y += self.speed
        if self.rect.top > 600:
            self.reset()


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 5

Now, the danger. Below the hit sound, load lose_sound, the sound of losing a life. It will play every time an enemy beats you. In update, delete the two lines that sent the enemy back up when it left through the bottom. If update reset it, the loop would never see it escape, and escaping now costs a life. Below score, add lives, set to three. Those are the lives each game starts with. And in the collisions, add an elif: if the enemy was not hit by a bullet, but got past the bottom, or its Rect touches the UFO, it goes back up, you lose a life, and the new sound plays. To check against a single Rect, like the UFO's, you use colliderect, which answers true or false.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-3.py`):

```
import random

import pygame

pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
```

```
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 6

One thing is missing: the ending. After the collisions, outside the for loop, check whether lives have reached zero, or gone below zero, which can happen if two enemies take lives in the same frame. If so, running becomes False, the loop ends, and the window closes. And the scoreboard now shows the lives too. In the same f-string, after the score, you add the word Lives and the value of the lives variable, so you can see how many you have left.

Your `game.py` now looks like this — highlighted lines are new or changed (file `step-4.py`):

```
import random

import pygame
```

```
pygame.init()
screen = pygame.display.set_mode((800, 600))
pygame.display.set_caption("Most Wanted UFO Mini")
clock = pygame.time.Clock()

background = pygame.image.load("assets/background_4.png").convert()
player_image = pygame.image.load("assets/player_1.png").convert_alpha()
bullet_image = pygame.image.load("assets/Bullet.gif").convert_alpha()
enemy_image = pygame.image.load("assets/enemy_1.png").convert_alpha()
shoot_sound = pygame.mixer.Sound("assets/shoot.wav")
hit_sound = pygame.mixer.Sound("assets/collision.wav")
lose_sound = pygame.mixer.Sound("assets/loselife.wav")
font = pygame.font.Font(None, 36)


class Enemy(pygame.sprite.Sprite):
    def __init__(self):
        super().__init__()
        self.image = enemy_image
        self.rect = self.image.get_rect()
        self.reset()

    def reset(self):
        self.rect.midbottom = (random.randint(20, 780), random.randint(-500, -40))
        self.speed = random.randint(1, 2)

    def update(self):
        self.rect.y += self.speed


player = player_image.get_rect(midbottom=(400, 590))
speed = 6
bullets = []
cooldown = 0
enemies = pygame.sprite.Group(Enemy() for _ in range(8))
score = 0
lives = 3

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT]:
        player.x -= speed
    if keys[pygame.K_RIGHT]:
        player.x += speed
    player.clamp_ip(screen.get_rect())

    if cooldown > 0:
        cooldown -= 1
    if keys[pygame.K_SPACE] and cooldown == 0:
        bullets.append(bullet_image.get_rect(midbottom=player.midtop))
        shoot_sound.play()
        cooldown = 15

    for bullet in bullets:
```

```
        bullet.y -= 10
    bullets = [b for b in bullets if b.bottom > 0]
    enemies.update()

    for enemy in enemies:
        hit = enemy.rect.collidelist(bullets)
        if hit != -1:
            bullets.pop(hit)
            enemy.reset()
            score += 10
            hit_sound.play()
        elif enemy.rect.top > 600 or enemy.rect.colliderect(player):
            enemy.reset()
            lives -= 1
            lose_sound.play()
    if lives <= 0:
        running = False

    screen.blit(background, (0, 0))
    enemies.draw(screen)
    for bullet in bullets:
        screen.blit(bullet_image, bullet)
    screen.blit(player_image, player)
    hud = font.render(f"Score: {score}   Lives: {lives}", True, (255, 255, 255))
    screen.blit(hud, (10, 10))
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Step 7

Run the game. Watch the scoreboard: the points go up with each hit. If an enemy gets past the bottom, you hear a different sound, and you lose a life. It is a real game now: it has rules, points, and a way to lose.

Try it:

```
python game.py
```

What you learned

Let's review. A collision is two Rects touching. collidelist searches a list, and colliderect compares against a single Rect. font.render turns text into an image. And lives give the game an ending. Your exercise: when you reach one hundred points, make the enemies fall faster. In the next lesson, the game gets a menu, a pause, and a game over screen.

- A collision is two Rects touching
- collidelist and colliderect
- font.render turns text into an image
- Lives lead to the end of the game

Summary

- `enemy.rect.collidelist(bullets)` returns the index of the first bullet it touches, or `-1` if it touches none.
- `enemy.rect.colliderect(player)` is `True` if the two `Rect`s overlap.
- `bullets.pop(hit)` removes the bullet that hit from the list.
- `font = pygame.font.Font(None, 36)` creates a font; `font.render(text, True, color)` returns an image with the text.
- `f"Score: {score}"` is an f-string: it puts the variable's value inside the text.
- When `lives` reaches 0, `running = False` ends the loop.

Exercise

Start from this lesson's `game.py`.

- When `score` reaches 100 or more, make new enemies fall at speed 2 to 4 instead of 1 to 2. Hint: `reset` can read the global `score` variable. Careful: `score` must exist before the enemies are created.
- Show lives as hearts: draw `assets/Life.png` once per life, instead of the `Lives` text.
- Bonus: every 5 hits in a row without losing a life, add 50 extra points.

Solution

```
life_image = pygame.image.load("assets/Life.png").convert_alpha()

score = 0   # move it BEFORE this line:
enemies = pygame.sprite.Group(Enemy() for _ in range(8))

# in Enemy.reset:
self.speed = random.randint(2, 4) if score >= 100 else random.randint(1, 2)

# when drawing, instead of the lives text:
hud = font.render(f"Score: {score}", True, (255, 255, 255))
screen.blit(hud, (10, 10))
for i in range(lives):
    screen.blit(life_image, (10 + i * 26, 44))
```

In this lesson's `game.py`, `score = 0` comes after the `Group`. Move it before: creating each `Enemy` calls `reset`, which already reads `score`.

Bonus:

```
streak = 0                  # next to score and lives
# on a hit:
streak += 1
if streak % 5 == 0:
    score += 50
# on losing a life:
streak = 0
```

Art and sounds: Most Wanted UFO Classic, by Jose Daniel Soto, licensed CC BY 4.0.
A Magic Lab Studio course · magiclabstudio.com